ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS LAB

EXAM-FRIENDLY SIMPLE C PROGRAMS

B.Tech II Year - I Semester
Geethanjali Institute of Science & Technology (Autonomous)
Department of Computer Science & Engineering

15 short source files Including matrix and list graph versions	**Easy to revise** Simple fixed examples and direct logic
Sample output Shown after each program	**Checked with GCC** C11, warnings enabled

The code is intentionally kept straightforward for exam revision. Fixed example inputs are used where they help shorten the program. The quick-sort/merge-sort times depend on the machine.

Contents

Experiment	Program
WEEK 01	AVL Tree: Insert, Delete and File In-order Traversal
WEEK 02	B-Tree of Order 5: Search, Insert and Delete
WEEK 03	Min-Heap and Max-Heap
WEEK 04A	BFS and DFS Using an Adjacency Matrix
WEEK 04B	BFS and DFS Using an Adjacency List
WEEK 05	Biconnected Components Using Tarjan DFS
WEEK 06	Quick Sort and Merge Sort Time Analysis
WEEK 07	Dijkstra: Adjacency Matrix vs Adjacency List
WEEK 08	Job Sequencing with Deadlines
WEEK 09	0/1 Knapsack Using Dynamic Programming
WEEK 10	N-Queens Using Backtracking
WEEK 11	0/1 Knapsack Using Backtracking
WEEK 12	Travelling Salesperson Using Branch and Bound
ADDITIONAL 13	Strassen Matrix Multiplication (2 x 2)
ADDITIONAL 14	m-Way Graph Coloring Using Backtracking

Compile any program: `gcc -std=c11 -Wall -Wextra -Wpedantic file.c -o file`
Run it with `./file` on Linux/WSL or `file.exe` on Windows.

WEEK 01 - PROGRAM 1

AVL Tree: Insert, Delete and File In-order Traversal

Reads integer keys from `in.txt`, deletes key 20 and saves the sorted traversal to `out.txt`.

Input file: Create `in.txt` containing `30 20 40 10 25 50`. The program deletes 20 and writes the traversal to `out.txt`.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int key, h;
    struct Node *left, *right;
} Node;
int height(Node *p) {
    return p ? p->h : 0;
}

int max(int a, int b) {
    return a > b ? a : b;
}

int balance(Node *p) {
    return p ? height(p->left) - height(p->right) : 0;
}

void update(Node *p) {
    p->h = 1 + max(height(p->left), height(p->right));
}

Node *rightRotate(Node *y) {
    Node *x = y->left, *t = x->right;
    x->right = y;
    y->left = t;
    update(y);
    update(x);
    return x;
}

Node *leftRotate(Node *x) {
    Node *y = x->right, *t = y->left;
    y->left = x;
    x->right = t;
    update(x);
    update(y);
    return y;
}

Node *insert(Node *p, int key) {
    if (!p) {
        p = malloc(sizeof(Node));
        if (!p) {
            printf("Memory error\n");
            exit(1);
        }
        p->key = key;
        p->h = 1;
        p->left = p->right = NULL;
        return p;
    }
    if (key < p->key) p->left = insert(p->left, key);
    else if (key > p->key) p->right = insert(p->right, key);
    else return p;
    update(p);
    int b = balance(p);
    if (b > 1 && key < p->left->key) return rightRotate(p);
    if (b < -1 && key > p->right->key) return leftRotate(p);
    if (b > 1 && key > p->left->key) {
        p->left = leftRotate(p->left);
        return rightRotate(p);
    }
    if (b < -1 && key < p->right->key) {
        p->right = rightRotate(p->right);
        return leftRotate(p);
    }
    return p;
}

Node *minNode(Node *p) {
    while (p->left) p = p->left;
    return p;
}

Node *deleteKey(Node *p, int key) {
```

```
    if (!p) return NULL;
    if (key < p->key) p->left = deleteKey(p->left, key);
    else if (key > p->key) p->right = deleteKey(p->right, key);
    else {
        if (!p->left || !p->right) {
            Node *child = p->left ? p->left : p->right;
            free(p);
            return child;
        }
        Node *q = minNode(p->right);
        p->key = q->key;
        p->right = deleteKey(p->right, q->key);
    }
    update(p);
    int b = balance(p);
    if (b > 1 && balance(p->left) >= 0) return rightRotate(p);
    if (b > 1 && balance(p->left) < 0) {
        p->left = leftRotate(p->left);
        return rightRotate(p);
    }
    if (b < -1 && balance(p->right) <= 0) return leftRotate(p);
    if (b < -1 && balance(p->right) > 0) {
        p->right = rightRotate(p->right);
        return leftRotate(p);
    }
    return p;
}

void inorder(Node *p, FILE *f) {
    if (p) {
        inorder(p->left, f);
        fprintf(f, "%d ", p->key);
        inorder(p->right, f);
    }
}

void freeTree(Node *p) {
    if (p) {
        freeTree(p->left);
        freeTree(p->right);
        free(p);
    }
}

int main(void) {
    FILE *in = fopen("in.txt", "r"), *out = fopen("out.txt", "w");
    Node *root = NULL;
    int x;
    if (!in || !out) {
        printf("Create in.txt and check file permissions.\n");
        if (in) fclose(in);
        if (out) fclose(out);
        return 1;
    }
    while (fscanf(in, "%d", &x) == 1) root = insert(root, x);
    root = deleteKey(root, 20);
    inorder(root, out);
    fprintf(out, "\n");
    fclose(in);
    fclose(out);
    freeTree(root);
    printf("Deleted 20. In-order traversal saved in out.txt.\n");
    return 0;
}
```

SAMPLE OUTPUT

```
Deleted 20. In-order traversal saved in out.txt.
File out.txt: 10 25 30 40 50
```

WEEK 02 - PROGRAM 2

B-Tree of Order 5: Search, Insert and Delete

Uses a small fixed example so the B-Tree operations are easier to follow.

Exam note: To keep this version short, deletion collects the existing keys and rebuilds the B-Tree without the requested key. It produces a valid resulting tree for the example, but it is not the standard borrow/merge deletion algorithm and does not have the usual deletion complexity.

SAMPLE OUTPUT

```
B-tree in sorted order: 5 6 7 10 12 17 20 30
Search 12: Found
After deleting 12: 5 6 7 10 17 20 30
```

SOURCE CODE

```
/* B-tree order 5: maximum four keys per node.
   To keep this exam version short, deletion rebuilds the tree. */
#include <stdio.h>
#include <stdlib.h>
#define MAX 4
typedef struct Node {
    int n, leaf, key[5];
    struct Node *c[6];
} Node;
Node *newNode(int leaf) {
    Node *p = calloc(1, sizeof(Node));
    if (!p) {
        printf("Memory error\n");
        exit(1);
    }
    p->leaf = leaf;
    return p;
}

Node *search(Node *p, int k) {
    if (!p) return NULL;
    int i = 0;
    while (i < p->n && k > p->key[i]) i++;
    if (i < p->n && k == p->key[i]) return p;
    return p->leaf ? NULL : search(p->c[i], k);
}

typedef struct {
    int split, key;
    Node *right;
} Split;
Split insertRec(Node *p, int k) {
    Split result = {0, 0, NULL};
    int i = p->n - 1;
    if (p->leaf) {
        while (i >= 0 && p->key[i] > k) {
            p->key[i+1] = p->key[i];
            i--;
        }
        p->key[i+1] = k;
        p->n++;
    }
    else {
        while (i >= 0 && k < p->key[i]) i--;
        i++;
        Split child = insertRec(p->c[i], k);
        if (child.split) {
            for (int j = p->n; j > i; j--) p->key[j] = p->key[j-1];
            for (int j = p->n+1; j > i+1; j--) p->c[j] = p->c[j-1];
            p->key[i] = child.key;
            p->c[i+1] = child.right;
            p->n++;
        }
    }
    if (p->n <= MAX) return result;
    Node *right = newNode(p->leaf);
    right->n = 2;
    right->key[0] = p->key[3];
    right->key[1] = p->key[4];
    if (!p->leaf) {
        right->c[0] = p->c[3];
        right->c[1] = p->c[4];
        right->c[2] = p->c[5];
        p->c[3] = p->c[4] = p->c[5] = NULL;
    }
    result.split = 1;
    result.key = p->key[2];
    result.right = right;
```

```
    p->n = 2;
    return result;
}

Node *insert(Node *root, int k) {
    if (search(root, k)) return root;
    Split result = insertRec(root, k);
    if (result.split) {
        Node *newRoot = newNode(0);
        newRoot->n = 1;
        newRoot->key[0] = result.key;
        newRoot->c[0] = root;
        newRoot->c[1] = result.right;
        return newRoot;
    }
    return root;
}

void collect(Node *p, int a[], int *n) {
    if (!p) return;
    for (int i = 0; i < p->n; i++) {
        if (!p->leaf) collect(p->c[i], a, n);
        a[(*n)++] = p->key[i];
    }
    if (!p->leaf) collect(p->c[p->n], a, n);
}

void freeTree(Node *p) {
    if (!p) return;
    if (!p->leaf) for (int i = 0; i <= p->n; i++) freeTree(p->c[i]);
    free(p);
}

Node *deleteKey(Node *root, int k) {
    int a[1000], n = 0, found = 0;
    collect(root, a, &n);
    freeTree(root);
    root = newNode(1);
    for (int i = 0; i < n; i++) {
        if (a[i] == k && !found) {
            found = 1;
            continue;
        }
        root = insert(root, a[i]);
    }
    return root;
}

void display(Node *p) {
    if (!p) return;
    int i;
    for (i = 0; i < p->n; i++) {
        if (!p->leaf) display(p->c[i]);
        printf("%d ", p->key[i]);
    }
    if (!p->leaf) display(p->c[i]);
}

int main(void) {
    Node *root = newNode(1);
    int a[] = {10, 20, 5, 6, 12, 30, 7, 17};
    for (int i = 0; i < 8; i++) root = insert(root, a[i]);
    printf("B-tree in sorted order: ");
    display(root);
    printf("\n");
    printf("Search 12: %s\n", search(root, 12) ? "Found" : "Not Found");
    root = deleteKey(root, 12);
    printf("After deleting 12: ");
    display(root);
    printf("\n");
    freeTree(root);
    return 0;
}
```

WEEK 03 - PROGRAM 3

Min-Heap and Max-Heap

Builds both heaps, deletes one key from each, and displays the arrays.

SOURCE CODE

```
#include <stdio.h>
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void heapify(int a[], int n, int i, int isMax) {
    int best = i, l = 2*i+1, r = 2*i+2;
    if (l < n && (isMax ? a[l] > a[best] : a[l] < a[best])) best = l;
    if (r < n && (isMax ? a[r] > a[best] : a[r] < a[best])) best = r;
    if (best != i) {
        swap(&a[i], &a[best]);
        heapify(a, n, best, isMax);
    }
}

void build(int a[], int n, int isMax) {
    for (int i = n/2-1; i >= 0; i--) heapify(a, n, i, isMax);
}

void deleteKey(int a[], int *n, int key, int isMax) {
    int i = 0;
    while (i < *n && a[i] != key) i++;
    if (i == *n) return;
    a[i] = a[--(*n)];
    build(a, *n, isMax);
}

void display(int a[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

int main(void) {
    int a[] = {10, 40, 20, 50, 30},
    b[] = {10, 40, 20, 50, 30},
    n = 5, m = 5;
    build(a, n, 1);
    printf("Max heap: ");
    display(a, n);
    deleteKey(a, &n, 40, 1);
    printf("After deleting 40: ");
    display(a, n);
    build(b, m, 0);
    printf("Min heap: ");
    display(b, m);
    deleteKey(b, &m, 20, 0);
    printf("After deleting 20: ");
    display(b, m);
    return 0;
}
```

SAMPLE OUTPUT

```
Max heap: 50 40 20 10 30
After deleting 40: 50 30 20 10
Min heap: 10 30 20 50 40
After deleting 20: 10 30 40 50
```

WEEK 04A - PROGRAM 4

BFS and DFS Using an Adjacency Matrix

Performs depth-first search and breadth-first search from vertex 0.

SOURCE CODE

```
#include <stdio.h>
int G[4][4] = {{0, 1, 1, 0}, {0, 0, 1, 0}, {1, 0, 0, 1}, {0, 0, 0, 0}};

int visited[4], q[4];
void dfs(int u) {
    printf("%d ", u);
    visited[u] = 1;
    for (int v = 0; v < 4; v++) if (G[u][v] && !visited[v]) dfs(v);
}

void bfs(int s) {
    int front = 0, rear = 0;
    for (int i = 0; i < 4; i++) visited[i] = 0;
    q[rear++] = s;
    visited[s] = 1;
    while (front < rear) {
        int u = q[front++];
        printf("%d ", u);
        for (int v = 0; v < 4; v++) if (G[u][v] && !visited[v]) {
            visited[v] = 1;
            q[rear++] = v;
        }
    }
}

int main(void) {
    printf("DFS: ");
    dfs(0);
    printf("\n");
    printf("BFS: ");
    bfs(0);
    printf("\n");
    return 0;
}
```

SAMPLE OUTPUT

```
DFS: 0 1 2 3
BFS: 0 1 2 3
```

WEEK 04B - PROGRAM 5

BFS and DFS Using an Adjacency List

Stores the graph as linked adjacency lists and traverses from vertex 2.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int v;
    struct Node *next;
} Node;
Node *adj[4];
int visited[4], q[4];
void addEdge(int u, int v) {
    Node *p = malloc(sizeof(Node));
    if (!p) exit(1);
    p->v = v;
    p->next = adj[u];
    adj[u] = p;
}

void dfs(int u) {
    printf("%d ", u);
    visited[u] = 1;
    for (Node *p = adj[u]; p; p = p->next) if (!visited[p->v]) dfs(p->v);
}

void bfs(int s) {
    int front = 0, rear = 0;
    for (int i = 0; i < 4; i++) visited[i] = 0;
    visited[s] = 1;
    q[rear++] = s;
    while (front < rear) {
        int u = q[front++];
        printf("%d ", u);
        for (Node *p = adj[u]; p; p = p->next) if (!visited[p->v]) {
            visited[p->v] = 1;
            q[rear++] = p->v;
        }
    }
}

void clear(void) {
    for (int i = 0; i < 4; i++) while (adj[i]) {
        Node *p = adj[i];
        adj[i] = p->next;
        free(p);
    }
}

int main(void) {
    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 2);
    addEdge(2, 0);
    addEdge(2, 3);
    printf("DFS: ");
    dfs(2);
    printf("\n");
    printf("BFS: ");
    bfs(2);
    printf("\n");
    clear();
    return 0;
}
```

SAMPLE OUTPUT

```
DFS: 2 3 0 1
BFS: 2 3 0 1
```

WEEK 05 - PROGRAM 6

Biconnected Components Using Tarjan DFS

Uses discovery time, low-link values and an edge stack.

SOURCE CODE

```
#include <stdio.h>
#define V 5
int G[V][V], disc[V], low[V], parent[V], timeCount;
int edgeU[20], edgeV[20], top = -1;
void printComponent(int u, int v) {
    printf("BCC: ");
    while (top >= 0) {
        int a = edgeU[top], b = edgeV[top--];
        printf("(%d-%d) ", a, b);
        if (a == u && b == v) break;
    }
    printf("\n");
}

void dfs(int u) {
    disc[u] = low[u] = ++timeCount;
    int children = 0;
    for (int v = 0; v < V; v++) if (G[u][v]) {
        if (!disc[v]) {
            children++;
            parent[v] = u;
            edgeU[++top] = u;
            edgeV[top] = v;
            dfs(v);
            if (low[v] < low[u]) low[u] = low[v];
            if ((parent[u] == -1 && children > 1) ||
                (parent[u] != -1 && low[v] >= disc[u]))
                printComponent(u, v);
        }
        else if (v != parent[u] && disc[v] < disc[u]) {
            if (disc[v] < low[u]) low[u] = disc[v];
            edgeU[++top] = u;
            edgeV[top] = v;
        }
    }
}

int main(void) {
    int e[][2] = {{0, 1}, {1, 2}, {2, 0}, {1, 3}, {3, 4}};
    for (int i = 0; i < 5; i++) G[e[i][0]][e[i][1]] = G[e[i][1]][e[i][0]] = 1;
    for (int i = 0; i < V; i++) parent[i] = -1;
    dfs(0);
    if (top >= 0) printComponent(-1, -1);
    return 0;
}
```

SAMPLE OUTPUT

```
BCC: (3-4)
BCC: (1-3)
BCC: (2-0) (1-2) (0-1)
```

WEEK 06 - PROGRAM 7

Quick Sort and Merge Sort Time Analysis

Sorts the same generated input with both algorithms. Tick counts vary by computer.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void quickSort(int a[], int low, int high) {
    if (low >= high) return;
    int pivot = a[high], i = low-1;
    for (int j = low; j < high; j++) if (a[j] <= pivot) swap(&a[++i], &a[j]);
    swap(&a[i+1], &a[high]);
    int p = i+1;
    quickSort(a, low, p-1);
    quickSort(a, p+1, high);
}

void mergeSort(int a[], int low, int high) {
    if (low >= high) return;
    int mid = (low+high)/2;
    mergeSort(a, low, mid);
    mergeSort(a, mid+1, high);
    int temp[1000], i = low, j = mid+1, k = 0;
    while (i <= mid && j <= high) temp[k++] = (a[i] < a[j])?a[i++]:a[j++];
    while (i <= mid) temp[k++] = a[i++];
    while (j <= high) temp[k++] = a[j++];
    for (i = low, k = 0; i <= high; i++) a[i] = temp[k++];
}

int main(void) {
    int a[1000], b[1000], n = 1000;
    clock_t t1, t2;
    srand(1);
    for (int i = 0; i < n; i++) a[i] = b[i] = rand()%10000;
    t1 = clock();
    quickSort(a, 0, n-1);
    t1 = clock()-t1;
    t2 = clock();
    mergeSort(b, 0, n-1);
    t2 = clock()-t2;
    printf("Quick Sort ticks: %ld\nMerge Sort ticks: %ld\n", (long)t1, (long)t2);
    printf("Both arrays sorted successfully.\n");
    return 0;
}
```

SAMPLE OUTPUT

```
Quick Sort ticks: (machine-dependent)
Merge Sort ticks: (machine-dependent)
Both arrays sorted successfully.
```

Note: exact tick counts vary with the computer and compiler; the important result is that both arrays are sorted.

WEEK 07 - PROGRAM 8

Dijkstra: Adjacency Matrix vs Adjacency List

Calculates shortest distances from source vertex 0 using both representations.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#define V 4
#define INF 9999

typedef struct Edge {
    int to, w;
    struct Edge *next;
} Edge;

int G[V][V] = {{0, 4, 1, 0}, {0, 0, 0, 1},
               {0, 2, 0, 5}, {0, 0, 0, 0}};
Edge *adj[V];

void addEdge(int u, int v, int w) {
    Edge *p = malloc(sizeof(Edge));
    if (!p) exit(1);
    p->to = v;
    p->w = w;
    p->next = adj[u];
    adj[u] = p;
}

void dijkstra(int useList) {
    int d[V], used[V] = {0};
    for (int i = 0; i < V; i++) d[i] = INF;
    d[0] = 0;

    for (int c = 0; c < V; c++) {
        int u = -1;
        for (int i = 0; i < V; i++)
            if (!used[i] && (u == -1 || d[i] < d[u])) u = i;
        if (u == -1 || d[u] == INF) break;
        used[u] = 1;

        if (!useList) {
            for (int v = 0; v < V; v++)
                if (G[u][v] && d[u] + G[u][v] < d[v])
                    d[v] = d[u] + G[u][v];
        } else {
            for (Edge *p = adj[u]; p; p = p->next)
                if (d[u] + p->w < d[p->to]) d[p->to] = d[u] + p->w;
        }
    }

    printf(useList ? "List:   " : "Matrix: ");
    for (int i = 0; i < V; i++) {
        if (d[i] == INF) printf("0->%d: INF  ", i);
        else printf("0->%d: %d  ", i, d[i]);
    }
    printf("\n");
}

int main(void) {
    addEdge(0, 1, 4);
    addEdge(0, 2, 1);
    addEdge(2, 1, 2);
    addEdge(1, 3, 1);
    addEdge(2, 3, 5);
    dijkstra(0);
    dijkstra(1);
    for (int i = 0; i < V; i++) while (adj[i]) {
        Edge *p = adj[i];
        adj[i] = p->next;
        free(p);
    }
    return 0;
}
```

SAMPLE OUTPUT

```
Matrix: 0->0: 0  0->1: 3  0->2: 1  0->3: 4
List:   0->0: 0  0->1: 3  0->2: 1  0->3: 4
```

WEEK 08 - PROGRAM 9

Job Sequencing with Deadlines

Greedy method: sort jobs by profit and place each job in the latest free slot.

SOURCE CODE

```
#include <stdio.h>

typedef struct {
    char id;
    int deadline, profit;
} Job;

int main(void) {
    Job j[] = {{'A', 2, 100}, {'B', 1, 19}, {'C', 2, 27},
               {'D', 1, 25}, {'E', 3, 15}};
    int n = 5, slot[5] = {0}, profit = 0;
    char result[5];

    /* Sort jobs by decreasing profit. */
    for (int i = 0; i < n-1; i++) {
        for (int k = i+1; k < n; k++) {
            if (j[i].profit < j[k].profit) {
                Job t = j[i];
                j[i] = j[k];
                j[k] = t;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        int last = (j[i].deadline < n) ? j[i].deadline : n;
        for (int k = last-1; k >= 0; k--) {
            if (!slot[k]) {
                slot[k] = 1;
                result[k] = j[i].id;
                profit += j[i].profit;
                break;
            }
        }
    }

    printf("Scheduled jobs: ");
    for (int i = 0; i < n; i++)
        if (slot[i]) printf("%c ", result[i]);
    printf("\nMaximum profit: %d\n", profit);
    return 0;
}
```

SAMPLE OUTPUT

```
Scheduled jobs: C A E
Maximum profit: 142
```

WEEK 09 - PROGRAM 10

0/1 Knapsack Using Dynamic Programming

Uses a DP table to calculate the maximum profit and selected weights.

SOURCE CODE

```
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main(void) {
    int w[] = {0, 2, 3, 4, 5};
    int p[] = {0, 3, 4, 5, 6};
    int n = 4, W = 5, dp[5][6] = {0};

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            if (w[i] <= j)
                dp[i][j] = max(dp[i-1][j], p[i] + dp[i-1][j-w[i]]);
            else
                dp[i][j] = dp[i-1][j];
        }
    }

    printf("Maximum profit: %d\nSelected weights: ", dp[n][W]);
    for (int i = n, j = W; i > 0 && j > 0; i--) {
        if (dp[i][j] != dp[i-1][j]) {
            printf("%d ", w[i]);
            j -= w[i];
        }
    }
    printf("\n");
    return 0;
}
```

SAMPLE OUTPUT

```
Maximum profit: 7
Selected weights: 3 2
```

WEEK 10 - PROGRAM 11

N-Queens Using Backtracking

Prints all solutions for a 4 x 4 board.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#define N 4
int x[N];
int safe(int row, int col) {
    for (int i = 0; i < row; i++) if (x[i] == col || abs(x[i]-col) == abs(i-row)) return 0;
    return 1;
}

void solve(int row) {
    if (row == N) {
        printf("Solution: ");
        for (int i = 0; i < N; i++) printf("%d ", x[i]+1);
        printf("\n");
        return;
    }
    for (int col = 0; col < N; col++) if (safe(row, col)) {
        x[row] = col;
        solve(row+1);
    }
}

int main(void) {
    solve(0);
    return 0;
}
```

SAMPLE OUTPUT

```
Solution: 2 4 1 3
Solution: 3 1 4 2
```

WEEK 11 - PROGRAM 12

0/1 Knapsack Using Backtracking

Tries including and excluding each item and remembers the best selection.

SOURCE CODE

```
#include <stdio.h>
#define N 4
int w[N] = {2, 3, 4, 5},
p[N] = {3, 4, 5, 6},
W = 5, best = 0, x[N], bestX[N];
void knapsack(int i, int weight, int profit) {
    if (weight > W) return;
    if (i == N) {
        if (profit > best) {
            best = profit;
            for (int k = 0; k < N; k++) bestX[k] = x[k];
        }
        return;
    }
    x[i] = 1;
    knapsack(i+1, weight+w[i], profit+p[i]);
    x[i] = 0;
    knapsack(i+1, weight, profit);
}

int main(void) {
    knapsack(0, 0, 0);
    printf("Maximum profit: %d\nChosen items: ", best);
    for (int i = 0; i < N; i++) printf("%d ", bestX[i]);
    printf("\n");
    return 0;
}
```

SAMPLE OUTPUT

```
Maximum profit: 7
Chosen items: 1 1 0 0
```

WEEK 12 - PROGRAM 13

Travelling Salesperson Using Branch and Bound

Searches tours while pruning branches whose lower bound cannot beat the best cost.

SOURCE CODE

```
#include <stdio.h>
#define N 4
#define INF 99999
int cost[N][N] = {{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30}, {20, 25, 30, 0}};

int visited[N], path[N], bestPath[N], best = INF;
int bound(int currentCost, int current) {
    int b = currentCost, m = INF;
    for (int j = 0; j < N; j++) {
        if (j != current && cost[current][j] < m)
            m = cost[current][j];
    }
    b += m;
    for (int i = 0; i < N; i++) if (!visited[i]) {
        m = INF;
        for (int j = 0; j < N; j++) if (i != j && cost[i][j] < m) m = cost[i][j];
        b += m;
    }
    return b;
}

void tsp(int city, int level, int currentCost) {
    if (level == N) {
        int total = currentCost+cost[city][0];
        if (total < best) {
            best = total;
            for (int i = 0; i < N; i++) bestPath[i] = path[i];
        }
        return;
    }
    for (int next = 1; next < N; next++) if (!visited[next]) {
        visited[next] = 1;
        path[level] = next;
        int newCost = currentCost+cost[city][next];
        if (bound(newCost, next) < best) tsp(next, level+1, newCost);
        visited[next] = 0;
    }
}

int main(void) {
    visited[0] = 1;
    path[0] = 0;
    tsp(0, 1, 0);
    printf("Minimum cost: %d\nTour: ", best);
    for (int i = 0; i < N; i++) printf("%d -> ", bestPath[i]);
    printf("0\n");
    return 0;
}
```

SAMPLE OUTPUT

```
Minimum cost: 80
Tour: 0 -> 1 -> 3 -> 2 -> 0
```

ADDITIONAL 13 - PROGRAM 14

Strassen Matrix Multiplication (2 x 2)

Demonstrates Strassen’s seven-product formula for two 2 x 2 matrices.

SOURCE CODE

```
#include <stdio.h>
int main(void) {
    int A[2][2] = {{1, 2}, {3, 4}},
    B[2][2] = {{5, 6}, {7, 8}},
    C[2][2];
    int M1 = (A[0][0]+A[1][1])*(B[0][0]+B[1][1]);
    int M2 = (A[1][0]+A[1][1])*B[0][0];
    int M3 = A[0][0]*(B[0][1]-B[1][1]);
    int M4 = A[1][1]*(B[1][0]-B[0][0]);
    int M5 = (A[0][0]+A[0][1])*B[1][1];
    int M6 = (A[1][0]-A[0][0])*(B[0][0]+B[0][1]);
    int M7 = (A[0][1]-A[1][1])*(B[1][0]+B[1][1]);
    C[0][0] = M1+M4-M5+M7;
    C[0][1] = M3+M5;
    C[1][0] = M2+M4;
    C[1][1] = M1-M2+M3+M6;
    printf("Product:\n%d %d\n%d %d\n", C[0][0], C[0][1], C[1][0], C[1][1]);
    return 0;
}
```

SAMPLE OUTPUT

```
Product:
19 22
43 50
```

ADDITIONAL 14 - PROGRAM 15

m-Way Graph Coloring Using Backtracking

Assigns one of three colors so adjacent vertices have different colors.

SOURCE CODE

```
#include <stdio.h>
#define N 4
#define M 3
int G[N][N] = {{0, 1, 1, 1}, {1, 0, 1, 0}, {1, 1, 0, 1}, {1, 0, 1, 0}},
color[N];
int safe(int v, int c) {
    for (int i = 0; i < N; i++) if (G[v][i] && color[i] == c) return 0;
    return 1;
}

int solve(int v) {
    if (v == N) return 1;
    for (int c = 1; c <= M; c++) if (safe(v, c)) {
        color[v] = c;
        if (solve(v+1)) return 1;
        color[v] = 0;
    }
    return 0;
}

int main(void) {
    if (solve(0)) {
        for (int i = 0; i < N; i++) printf("V%d = C%d\n", i, color[i]);
    }
    else printf("No coloring possible with %d colors.\n", M);
    return 0;
}
```

SAMPLE OUTPUT

```
V0 = C1
V1 = C2
V2 = C3
V3 = C2
```